Hide menu
Introduction to Ensembles: Bagging
Introduction to Boosting Ensembles
Boosting Algorithms
Slide Decks
Assessments
Quiz: Week 5 Quiz
Submitted
Programming Assignment: Week 5 : Ensembles
3h
Peer-graded Assignment: Week 5: Ensembles
1h
Review Your Peers: Week 5: Ensembles

Peer-graded Assignment: Week 5: Ensembles

You passed!
Congratulations. You earned 46 / 46 points. Review the feedback below and continue the course when you are ready. You can also help more peers by reviewing their submissions.

Review assignments

Instructions My submission Discussions

Your fellow learners have submitted their reviews anonymously. All names are still visible to course instructors.

Ensembles Project

Submitted on November 19, 2023

PROMPT

Upload a **screenshot** of the **fit function** from the **class adaboost.**

AdaBoost fit

```
def fit(self, X_train, y_train):
    """
    Train AdaBoost classifier on data. Sets alphas and learners.

    Args:
        X_train (ndarray): [n_samples x n_features] ndarray of training data
        y_train (ndarray): [n_samples] ndarray of data
    """

    w = np.ones(len(y_train), dtype = np.float128)
    w /= np.sum(w)
    for k in range(self.n_learners):
        h = clone(self.base)
        h.fit(X_train, y_train, sample_weight=w)
        self.learners.append(h)

        y_pred = h.predict(X_train)

        #err_k = np.sum(w * (y_train != y_pred)) / np.sum(w)
        err_k = self.error_rate(y_train, y_pred, w)
        self.alpha[k] = 0.5 * np.log((1 - err_k) / err_k)

        w = w * np.exp(-self.alpha[k] * y_train * y_pred)
        #for i in range(len(y_train)):
        #    w[i] = w[i] * np.exp(-self.alpha[k] * y_train[i] * y_pred[i])
        w /= np.sum(w)

    return self
```

AdaBoost fit

RUBRIC

Here is a sample solution for the **fit function in the class AdaBoost:**

```
def fit(self, X_train, y_train):
        """
        Train AdaBoost classifier on data. Sets alphas and learners.

        Args:
            X_train (ndarray): [n_samples x n_features] ndarray of training data
            y_train (ndarray): [n_samples] ndarray of data
        """

        # =================================================================
        # TODO

        # Note: You can create and train a new instantiation
        # of your sklearn decision tree as follows
        # you don't have to use sklearn's fit function,
        # but it is probably the easiest way

        # w = np.ones(len(y_train))
        # h = clone(self.base)
        # h.fit(X_train, y_train, sample_weight=w)

        # =================================================================

        w = np.ones(len(y_train))
        w /= np.sum(w)
        for k in range(self.n_learners):
            h = clone(self.base)
            h.fit(X_train, y_train, sample_weight=w)
            y_pred = h.predict(X_train)
            err = self.error_rate(y_train, y_pred, w)
            self.alpha[k] = 0.5 * np.log((1 - err) / err)
            w *= np.exp(- self.alpha[k] * y_train * y_pred)
            w /= np.sum(w)
            self.learners.append(h)
```

Pay attention to this particular line:

```
h.fit(X_train, y_train, sample_weight=w)
```

The learner doesn't have to use sklearn's built-in fit function, but does the learner's code pass the correct data (training data) **and** weight the samples?

- **2 points**
  The learner did **both of the following**: passed the correct data (training data) **and** weighted the samples.
- 1 point
  The learner did **one of the following:** passed the correct data (training data) **or** weighted the samples.
- 0 points
  The learner **didn't do either of the following**: passed the correct data (training data) **or** weighted the samples**.**

In the sample fit function solution, these two lines are for the fit function to **calculate error** using the **error rate function**.

```
y_pred = h.predict(X_train)
err = self.error_rate(y_train, y_pred, w)
```

Within the **fit function**, does the learner properly calculate the error using the **error_rate function** in the **AdaBoost class**?

- **1 point**
  The learner properly calculates the error using the error_rate function in the fit function.
- 0 points
  The learner does not properly calculate the error using the error_rate function in the fit function.

The line below in the sample **fit function calculates alpha in the adaboost algorithm:**

```
 self.alpha[k] = 0.5 * np.log((1 - err) / err)
```

Does the learner's fit function properly calculate **alpha for the adaboost classifier**?

- **3 points**
  Fit function contains a calculation for alpha **and** alpha calculation is equivalent to the sample calculation**.**
- 1 point
  Fit function contains a calculation for alpha **but** alpha calculation is not equivalent to the sample calculation.
- 0 points
  Fit function does not contain a calculation for alpha.

The line below in the sample fit function, **implements the weights correctly:**

```
w *= np.exp(- self.alpha[k] * y_train * y_pred)
```

Does the learner's fit function **implement the weights correctly**?

- **3 points**
  Fit function implements weights **and** weights are implemented in an equivalent way to the sample fit function.
- 1 point
  Fit function implements weights **but** weights are not implemented in an equivalent way to the sample fit function.
- 0 points
  Fit function does not implement weights.

The following line **updates weights** according to the AdaBoost algorithm:

```
w /= np.sum(w)
```

Does the fit function update weights according to the AdaBoost algorithm?

- **2 points**
  Fit function updates weights **and** updates weighs in an equivalent way to the sample fit function.
- 1 point
  Fit function updates weights **but** does not update weights in an equivalent way to the sample fit function.
- 0 points
  Fit function does not update weights.

PROMPT

Upload a screenshot of the **error_rate function** from the **class adaboost**.

Ada Error Rate

```
def error_rate(self, y_true, y_pred, weights):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    return np.sum(weights * (y_true != y_pred)) / np.sum(weights)
```

Ada Error Rate

RUBRIC

In the sample solution, these two lines are for the fit function to **calculate error** using the **error rate function**.

```
y_pred = h.predict(X_train)
err = self.error_rate(y_train, y_pred, w)
```

Here is a **sample error rate function solution** for the adaboost class:

```
def error_rate(self, y_true, y_pred, weights):
    # Implement the weighted error rate
    return 1 - np.sum(weights * np.equal(y_true, y_pred)) / np.sum(weights)
```

- **2 points**
  The learner's error rate function calculates an equivalent formula to the formula in the sample error_rate function solution.
- 1 point
  The formula the learner utilizes for the error_rate function is not equivalent to the sample error_rate function solution.
- 0 points
  error_rate function not implemented.

PROMPT

Upload a **screenshot** of using the **fit function to fit the Adaboost classifier with 300 base decision stumps.**

AdaBoost Stumps

```
# use fit function to fit Adaboost classifier called clf with 300 base decision stumps

clf = AdaBoost(n_learners=300, base=DecisionTreeClassifier(max_depth=1))
clf.fit(data.x_train,data.y_train)
# clf.alpha
```

AdaBoost Stumps

RUBRIC

Here is a sample solution:

```
clf = AdaBoost(n_learners=300, base=DecisionTreeClassifier(max_depth=1))# TODO

clf.fit(data.x_train, data.y_train)

print(clf.alpha)
```

Did the learner pass the correct data for the fit function?

- **1 point**
  Learner's input to the AdaBoost classifier fit function was (data.x_train, data.y_train) .
- 0 points
  Learner's input to the AdaBoost classifier fit function was not (data.x_train, data.y_train).

PROMPT

Part C has the following directions:

Once your **predict function** is written up, you need to test the scores on the function. To do this compute the scores on the prediction in the **score function**. Use the **score function** to then complete **staged_score** to collect the scores for every boosting iterations. Plot the misclassification error for train and test sets (misclassification error = 1- score).

Upload a screenshot or copy of your **code for the misclassification error for the train and test sets**.

code for the misclassification error

```
clf = AdaBoost(n_learners=150, base=DecisionTreeClassifier(max_depth=1)).fit(data.x_train,data.y_train)
# compute staged accuracy scores
train_scores = clf.staged_score(data.x_train, data.y_train)
test_scores = clf.staged_score(data.x_test, data.y_test)

# plot misclassification error for train and test sets
plt.figure(figsize=(10,7))
plt.plot(range(clf.n_learners), 1 - train_scores, label = 'train')
plt.plot(range(clf.n_learners), 1 - test_scores, label = 'test')
plt.legend(fontsize=15)
plt.title('Misclassification error with AdaBoost', size=20)
plt.xlabel('iterations (# learner decision stumps)', size=20)
plt.ylabel('misclassification error', size=20)
plt.grid()
plt.show()
```

code for the misclassification error

RUBRIC

Here is a sample solution:

```
train_scores = clf.staged_score(data.x_train, data.y_train) # TODO
valid_scores = clf.staged_score(data.x_test, data.y_test) # TODO
fig, ax = plt.subplots(nrows=1, ncols=1, figsize=(10,4))
ax.plot(1 - train_scores, label="train")
ax.plot(1 - valid_scores, label="valid")
ax.set_xlabel("boosting iteration", fontsize=16)
ax.set_ylabel("misclassification error", fontsize=16)
ax.grid(alpha=0.25)
ax.legend(loc="upper right", fontsize=20)
```

Evaluate the learner's solution.

- **5 points**
  Includes all of the following: calls the staged_score function with data.x_train and data.y_train to collect training scores **and** calls the staged_score function with data.x_test and data.y_test to collect test scores **and** plots the misclassification error for the training set (1 - training scores) **and** plots the misclassification error for the test set (1 - test scores).
- 3 points
  Missing two of the following: calls the staged_score function with data.x_train and data.y_train to collect training scores **or** calls the staged_score function with data.x_test and data.y_test to collect test scores **or** plots the misclassification error for the training set (1 - training scores) **or** plots the misclassification error for the test set (1 - test scores).
- 0 points
  Missing all of the following: calls the staged_score function with data.x_train and data.y_train to collect training scores **or** calls the staged_score function with data.x_test and data.y_test to collect test scores **or** plots the misclassification error for the training set (1 - training scores) **or** plots the misclassification error for the test set (1 - test scores).

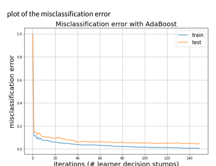
PROMPT

Part C has the following directions:

Once your **predict function** is written up, you need to test the scores on the function. To do this compute the scores on the prediction in the **score function**. Use the **score function** to then complete **staged_score** to collect the scores for every boosting iterations. Plot the misclassification error for train and test sets (misclassification error = 1- score).

Upload a screenshot or copy of your **plot of the misclassification error for the train and test sets**.

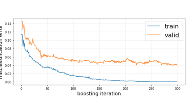
plot of the misclassification error



plot of the misclassification error

RUBRIC

Here is a sample solution:



Evaluate the learner's plot.

- **5 points**
  Misclassification plot includes the following: misclassification error on the y-axis **and** boosting iteration on the x-axis **and** plots 300 boosting iterations **and** plot includes training set curve and test set curves.
- 3 points
  Misclassification plot missing one of the following: misclassification error on the y-axis **or** boosting iteration on the x-axis **or** plots 300 boosting iterations **or** plot includes training set curve and test set curves.
- 0 points
  Not a misclassification plot or missing all of the following: misclassification error on the y-axis **and** boosting iteration on the x-axis **and** plots 300 boosting iterations **and** plot includes training set curve and test set curves.

PROMPT

Problem 2 Building a Random Forest Classifier to Classify MNIST Digits 3 and 8 has the following directions for Part A:

Complete the **create_tree function** to build a new tree trained on a subset of data. Within this function a decision tree classifier is built and trained on the subset of data with the subset of features.

Upload a screenshot of your create_tree function.

create_tree function

```
def create_tree(self,i):
    """
    create a single decision tree classifier
    """

    idxs = np.random.permutation(len(self.y))[:self.sample_sz]
    idxs = np.asarray(idxs)

    f_idxs = np.random.permutation(self.x.shape[1])[:self.n_features]
    f_idxs = np.asarray(f_idxs)


    if i==0:
        self.features_set = np.array(f_idxs, ndmin=2)
    else:
        self.features_set = np.append(self.features_set, np.array(f_idxs,ndmin=2),axis=0)

    # build a decision tree classifier and train it with x and y that is a subset of data (use idxs and f_idxs)

    clf = DecisionTreeClassifier(max_depth = self.max_depth, min_samples_leaf = self.min_samples_leaf)
    x, y = self.x[idxs[:, np.newaxis], f_idxs], self.y[idxs]
    clf.fit(x, y)

    return clf
```

create_tree function

RUBRIC

Here is a sample solution:

```
def create_tree(self,i):
        """
        create a single decision tree classifier
        """

        idxs = np.random.permutation(len(self.y))[:self.sample_sz]
        idxs = np.asarray(idxs)

        f_idxs = np.random.permutation(self.x.shape[1])[:self.n_features]
        f_idxs = np.asarray(f_idxs)


        if i==0:
            self.features_set = np.array(f_idxs, ndmin=2)
        else:
            self.features_set = np.append(self.features_set, np.array(f_idxs,ndmin=2),axis=0)

        # TODO: build a decision tree classifier and train it with x and y that is a subset of data (use idxs and f_idxs)



        clf = DecisionTreeClassifier(max_depth = self.max_depth, min_samples_leaf=self.min_samples_leaf)

        x, y = self.x[idxs][:,f_idxs], self.y[idxs]
        clf.fit(x,y)

        return clf
```

Evaluate the learner's code.

- **10 points**
  Includes all of the following: max_depth for the DecisionTreeClassifier is set to self.max_depth **and** min_samples_leaf is set to self.min_samples_leaf **and** fits classifier with (x,y) and x and y are subsets of the data using idxs and f_idxs.
- 6 points
  Missing one of the following: max_depth for the DecisionTreeClassifier is set to self.max_depth **or** min_samples_leaf is set to self.min_samples_leaf **or** fits classifier with (x,y) **or** x and y are subsets of the data using idxs and f_idxs.
- 3 points
  Missing two of the following: max_depth for the DecisionTreeClassifier is set to self.max_depth **or** min_samples_leaf is set to self.min_samples_leaf **or** fits classifier with (x,y) **or** x and y are subsets of the data using idxs and f_idxs.
- 0 points
  Missing all of the following: max_depth for the DecisionTreeClassifier is set to self.max_depth **and** min_samples_leaf is set to self.min_samples_leaf **and** fits classifier with (x,y) and x and y are subsets of the data using idxs and f_idxs.

PROMPT

Problem 2 Building a Random Forest Classifier to Classify MNIST Digits 3 and 8 has the following directions for Part B:

In this part you will have to complete three steps. The first step is:
1. Complete the **predict** function in RandomForest class so as to make predictions using just the features.

Upload a **screenshot of your predict function** here.

predict function

```
def predict(self, x):

    # create a vector of predictions  and return
    # return the predictions of the final ensembles based on the individual trees' predicitons
    from collections import Counter
    y_hats = []
    for i in range(len(x)):
        preds = [self.trees[j].predict(x[i, self.features_set[j]].reshape(1,-1))[0] for j in range(len(self.trees))
        # majority voting of the trees on behalf of the classes +1 and -1,
        # implemented using collections.Counter.most_common() method
        y_hats.append(Counter(preds).most_common(1)[0][0])
    return y_hats
```

predict function

RUBRIC

Here is a sample solution:

```
def predict(self, x):

        # TODO: create a vector of predictions  and return
        # You will have to return the predictions of the final ensembles based on the individual trees' predicitons


        ### BEGIN SOLUTION
        predict_list= [t.predict(x[:,self.features_set[i,:]]) for i, t in zip(range(self.features_set.shape[0]),self.trees) ]
        predict_array=np.array(predict_list).transpose()
        print(predict_array.shape)
        predict_minus1 = np.array((predict_array==-1).sum(axis=1)/predict_array.shape[1])
        predict_ = predict_minus1
        predict_ = np.empty(predict_minus1.shape)
        predict_[(predict_minus1 > 0.5)] = -1
        predict_[(predict_minus1 <= 0.5)] = 1

        return predict_
        ### END SOLUTION
```

Evaluate the learner's response. The learner's code does not need to exactly match the sample solution, but the solution is an example of meeting the grading criteria.

- **5 points**
  Includes all of the following: creates and returns a vector of predictions **and** predictions of the final ensemble are based on individual trees' predictions **and** code has equivalent of predict_minus1 in the sample solution.
- 3 points
  Missing one of the following: creates and returns a vector of predictions **or** predictions of the final ensemble are based on individual trees' predictions **or** code has equivalent of predict_minus1 in the sample solution.
- 1 point
  Missing two of the following: creates and returns a vector of predictions **or** predictions of the final ensemble are based on individual trees' predictions **or** code has equivalent of predict_minus1 in the sample solution.
- 0 points
  Missing all of the following: creates and returns a vector of predictions **and** predictions of the final ensemble are based on individual trees' predictions **and** code has equivalent of predict_minus1 in the sample solution.